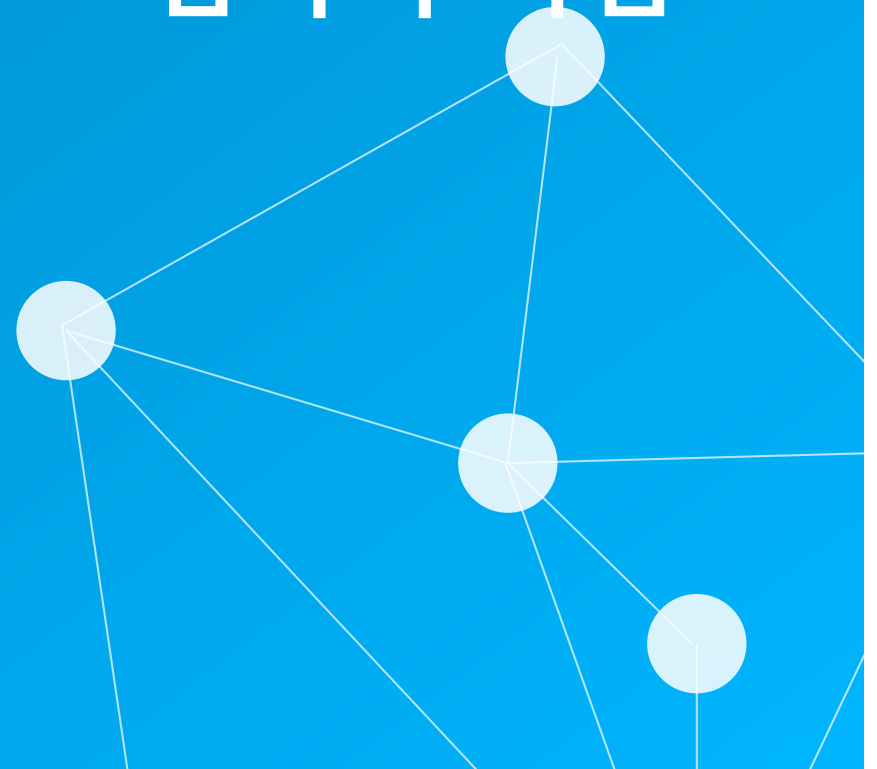


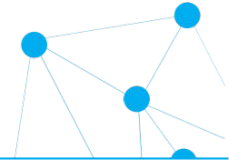
08

CHAPTER

탐욕적 기법



학습 내용



8.1 거스름돈 동전 최소화

8.2 분할 가능한 배낭 채우기

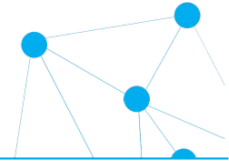
8.3 최소비용 신장트리: Prim 알고리즘

8.4 최소비용 신장트리: Kruskal 알고리즘

8.5 Dijkstra의 최단경로 알고리즘

8.6 허프만 코드

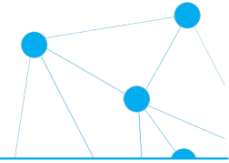
탐욕적 기법(greedy method)



- Greedy method
 - 어떤 결정을 해야 할 때 마다 “그 순간에 최적”이라고 생각되는 것을 선택 (“근시안적” 알고리즘)
 - 최적의 해답을 주는지 검증해야 함
- 탐욕적 기법이 사용될 수 있는 경우
 - 탐욕적 기법을 사용해도 항상 최적해를 구할 수 있는 문제
 - 최적해를 구하는 것이 현실적으로 불가능한 경우
 - 예) 분기 한정 기법(9장)에서 좋은 한계값을 구하기 위해 사용
- 알고리즘 설계 과정

- ① 현재 주어진 문제에 대한 가장 탐욕적인 해를 선택한다.
- ② 이 해가 가능한 해인지를 검사한다. 만약 가능하지 않다면 ①로 돌아가 다음으로 가장 탐욕적인 해를 선택한다.
- ③ 만약 가능한 해라면, 문제가 해결되었는지를 검사한다. 만약 아직 문제가 해결되지 않았으면 이 해를 적용해 크기가 줄어든 부분 문제에 대해 다시 ①로 돌아가 해를 구한다.

8.1 거스름돈 동전 최소화



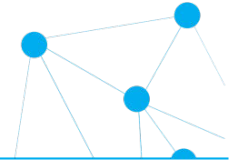
액면가가 서로 다른 m 가지의 동전 $\{C_1, C_2, \dots, C_m\}$ 이 있다. 거스름돈으로 V 원을 동전으로만 돌려주어야 한다면 최소 몇 개의 동전이 필요한지를 구하라. 단, 모든 동전은 무한히 사용할 수 있고, 동전들은 액수가 큰 것부터 내림차순으로 순서대로 정렬되어 있다.

- 우리나라 동전



- 620원: 500원 + 100원 + 10원 $\times$ 2 $\rightarrow$ 동전 4개
- 345원: 100원 $\times$ 3 + 10원 $\times$ 4 + 5원 $\rightarrow$ 동전 8개
- 572원: 500원 + 50원 + 10원 $\times$ 2 + 1원 $\times$ 2 $\rightarrow$ 동전 6개
- 탐욕적 전략
 - 가장 액수가 큰 동전부터 최대한 사용하면서 거스름돈을 맞춤
- 현재까지는 최적해. 만약 60원 동전이 생긴다면?
- 항상 최적해를 구하는 방법
 - 방법 1: 완전 탐색 $\rightarrow$ 지수함수
 - 방법 2: 동적 계획법 $\rightarrow O(mV)$

탐욕적 전략의 동전 최소화 알고리즘



- 탐욕적인 해: 액면가가 가장 큰 동전을 사용한다. 남은 잔돈을 가장 작게 만들어 전체 동전의 수를 최소화시킬 가능성이 가장 크기 때문이다.
- 가능한 해인지 검사: 사용한 동전이 잔돈을 초과하지 않아야 한다.

– 예: 580원 → 5개

$$500 * 1 + 100 * 0 + 50 * 1 + 10 * 3$$

알고리즘

알고리즘 8.1

거스름돈 최소화 알고리즘

```
01 def min_coins_greedy( coins, V ):
02     count = []
03     for i in range(len(coins)) :
04         cnt = V // coins[i]
05         count.append(cnt)
06         V -= cnt * coins[i]
07     return count
```

알고리즘 테스트

탐욕적 전략의 거스름돈 동전 최소화

```
coins = [500 , 100 , 50 , 10 , 5 , 1]
V = 580
print("잔돈 액수 = ", V)
print("동전 종류 = ", coins)
print("동전 개수 = ", min_coins_greedy(coins, V ))
```

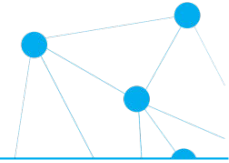
C:\WINDOWS\system32\cmd.exe

```
잔돈 액수 = 580
동전 종류 = [500, 100, 50, 10, 5, 1]
동전 개수 = [1, 0, 1, 3, 0, 0]
```

각 동전의 개수

- 복잡도: $O(m)$

최적해를 보장할까?



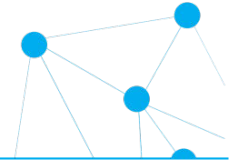
- 동전 체계에 따라 다름(항상 보장하지는 않음)

동전의 액면가 중에서 어떤 두 개를 고르더라도 큰 액면가를 작은 액면가로 나누어 떨어지는 동전 체계를 갖는다면 최적해를 보장한다. 작은 액면가를 여러 개 모으면 반드시 큰 액면가를 만들 수 있기 때문이다.

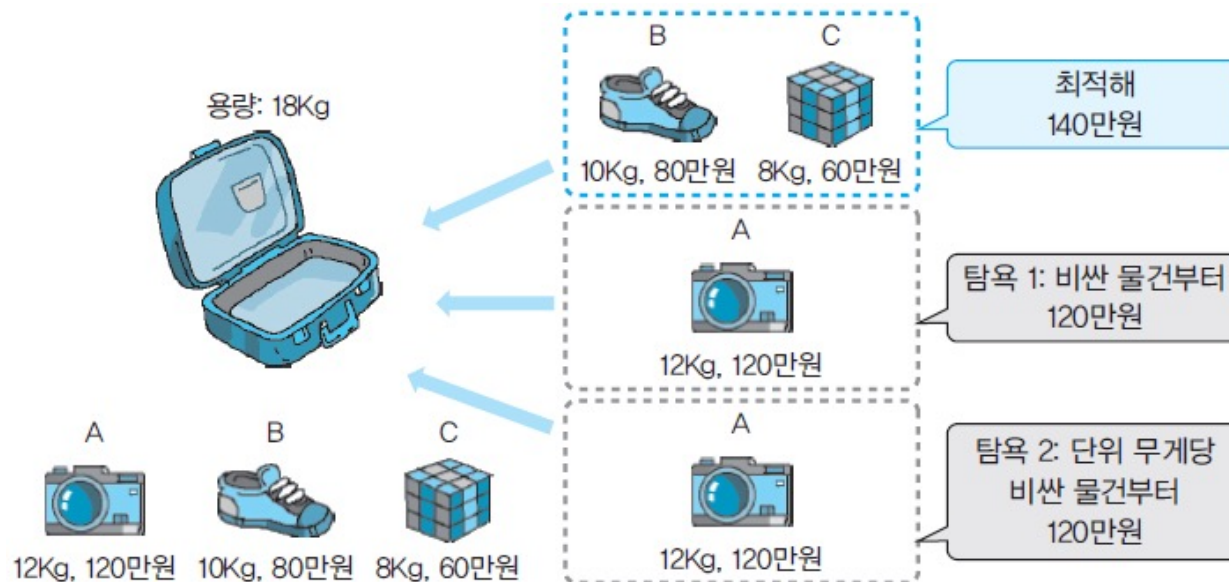
– 예: 우리나라에 60원 동전이 생긴다면?

- 120원? $100\text{원} \times 1 + 10\text{원} \times 2 \rightarrow 3\text{개}$
- 120원? $60\text{원} \times 2 \rightarrow 2\text{개}$

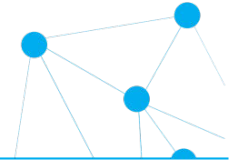
8.2 분할 가능한 배낭 채우기



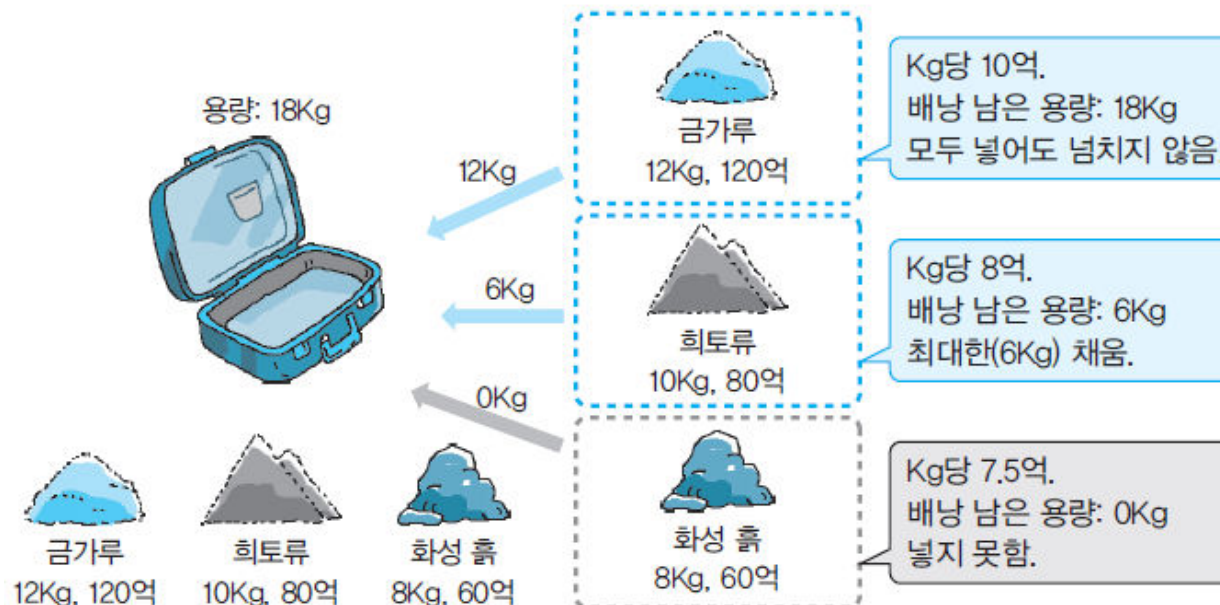
- 0-1 배낭 채우기 탐욕적 전략
 - 탐욕 1: 무게와 상관없이 가장 비싼 물건부터 넣어보는 방법
 - 탐욕 2: 단위 무게당 가격이 가장 높은 물건부터 넣어보는 방법
 - 탐욕 3: 가장 가벼운 물건부터 넣어보는 방법
- 0-1 배낭 채우기: 탐욕적 전략으로 최적해를 구하지 못함



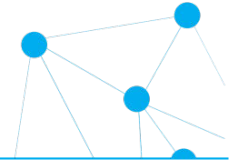
분할 가능한 배낭 채우기 문제



- 0-1 배낭 채우기 문제(문제 3.6)을 변형
 - 물건들은 분할해서 일부분만을 넣을 수도 있다.
- 탐욕적 기법
 - 탐욕적인 해: 무게당 가격이 비싼 물건부터 순서대로 배낭의 남은 용량을 넘지 않는 최대한으로 채움
 - 가능한 해 검사: 배낭의 용량을 넘치지만 않으면 항상 가능한 해



알고리즘



알고리즘 테스트

분할 가능한 배낭 채우기(탐욕적 기법)

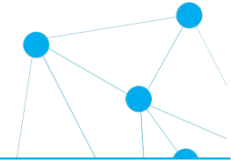
```
obj = [ ("A", 10, 80), ("B", 12, 120), ("C", 8, 60)]    # (물건,무게,가치)
print("W = 18 ", obj)
print("부분적인배낭(18): ", knapSack_fractional_greedy(obj,18), end='\n\n')
```

```
obj = [ ("A", 10, 60), ("B", 40, 40), ("C", 20, 100), ("D", 30, 120) ]
print("W = 50 ", obj)
print("부분적인배낭(50): ", knapSack_fractional_greedy(obj, 50))
```

```
C:\WINDOWS\system32\cmd.exe
W = 18 [ ('A', 10, 80), ('B', 12, 120), ('C', 8, 60)]
부분적인배낭(18): 168.0 120(B) + 6*80/10(A) + 0(C)
W = 50 [ ('A', 10, 60), ('B', 40, 40), ('C', 20, 100), ('D', 30, 120)]
부분적인배낭(50): 240.0 60(A) + 100(C) + 20*120/30(D) + 0(B)
```

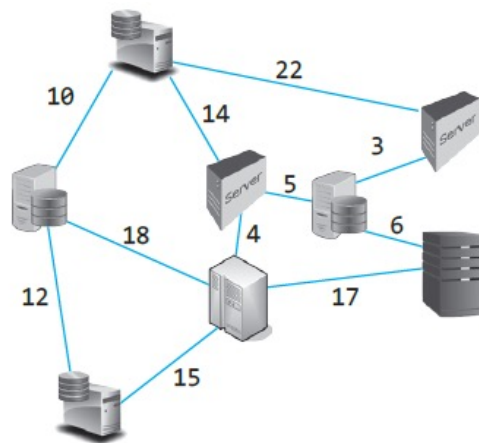
- 시간 복잡도: 정렬에 좌우 $\rightarrow O(n \log_2 n)$
- 항상 최적해를 구함

8.3 최소비용 신장트리: Prim

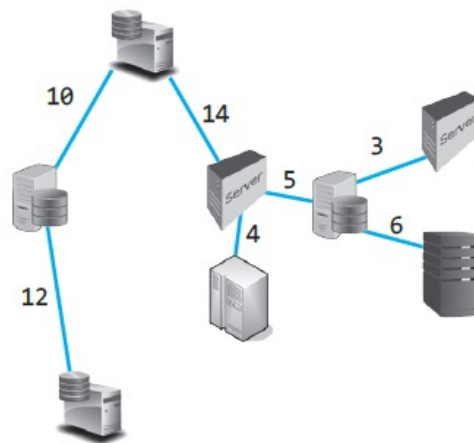


가중치 그래프 $G=(V, E)$ 가 주어졌다. G 의 모든 신장트리 중에서 간선들의 가중치 합이 최소인 신장트리를 찾아라. 이러한 트리를 최소비용 신장트리(MST: minimum spanning tree)라고 한다.

- 신장트리(spanning tree)
 - 그래프 내의 모든 정점을 포함하는 트리
- 최소비용 신장트리(MST)
 - “간선들의 비용 최소화”하는 신장트리



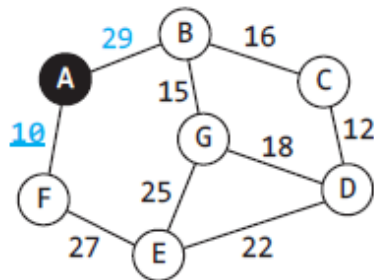
(a) 사이트 사이의 연결 비용



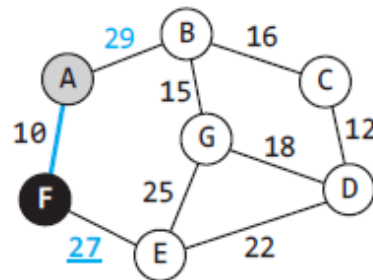
(b) 최소비용 신장트리의 예

Prim의 탐욕

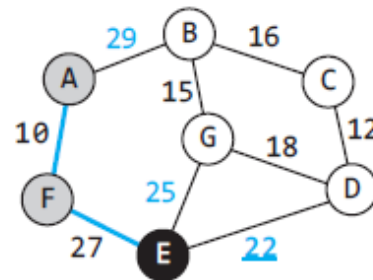
- 현재까지 만들어진 MST의 인접 정점들 중에서 간선 가중치가 최소인 정점을 반복적으로 선택해 트리를 확장



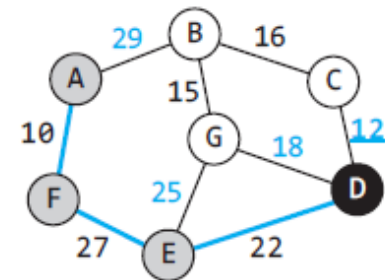
(a) 시작 정점 A 추가



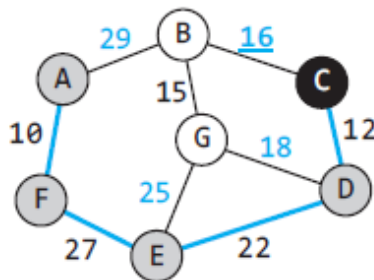
(b) 최소 간선 (A,F) 선택
정점 F 추가



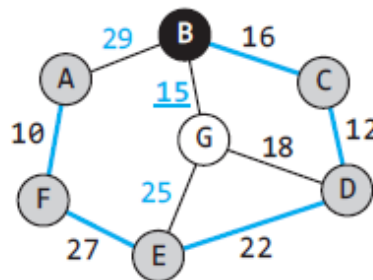
(c) 최소 간선 (E,F) 선택
정점 E 추가



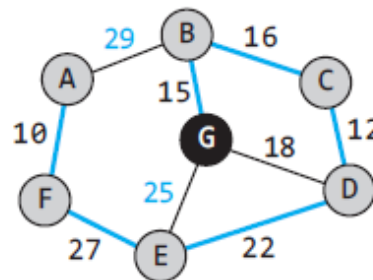
(d) 최소 간선 (E,D) 선택
정점 D 추가



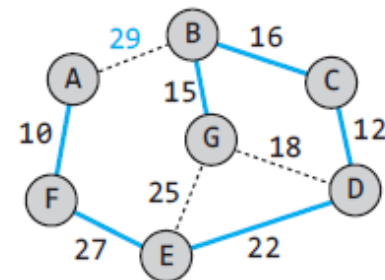
(e) 최소 간선 (C,D) 선택
정점 C 추가



(f) 최소 간선 (B,C) 선택
정점 B 추가

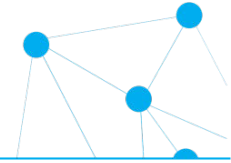


(g) 최소 간선 (B,G) 선택
정점 G 추가



(h) 최종 MST 완성

알고리즘 정확성



알고리즘 8.3

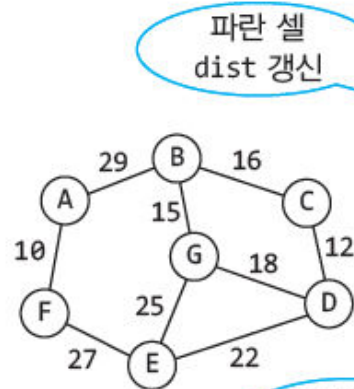
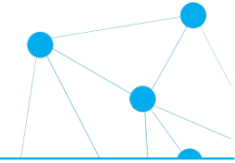
Prim의 최소비용 신장트리 알고리즘(자연어)

Prim()

1. 그래프에서 시작 정점을 선택하여 초기 트리를 만든다.
2. 현재 트리의 정점들과 인접한 정점들 중에서 간선의 가중치가 가장 작은 정점 v 를 선택한다.
3. 이 정점 v 와 이때의 간선을 트리에 추가한다.
4. 모든 정점이 삽입될 때 까지 2번으로 이동한다.

- Prim 알고리즘의 정확성(Correctness)
 - 심화학습: 333~334쪽

알고리즘: 가중치가 가장 작은 간선의 선택



인덱스	0	1	2	3	4	5	6
	A	B	C	D	E	F	G
dist	0	INF	INF	INF	INF	INF	INF
0	29	INF	INF	INF	10	10	INF
0	29	INF	INF	27	10	10	INF
0	29	INF	22	27	10	10	25
0	29	12	22	17	10	10	18
0	16	12	22	27	10	10	18
0	16	12	22	27	10	10	15
0	16	12	22	27	10	10	15

맨 처음: 시작 정점 A → dist[A]=0

최소 정점 A 선택 → dist[F, B] 갱신

최소 정점 F 선택 → dist[E] 갱신

최소 정점 E 선택 → dist[D, G] 갱신

최소 정점 D 선택 → dist[C, G] 갱신

최소 정점 C 선택 → dist[B] 갱신

최소 정점 B 선택 → dist[G] 갱신

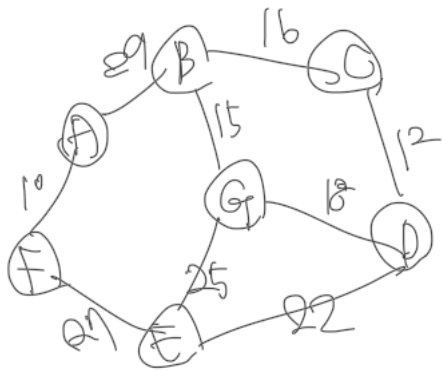
최소 정점 G 선택 → MST 완성

[그림 8.13] 최소 간선 선택을 위한 dist 배열 변화 과정

- getMinVertex()
 - dist[]
 - selected[]

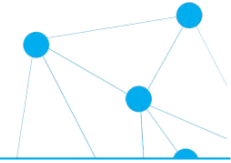
```

01 def getMinVertex(dist, selected) :
02     minv = -1
03     mindist = INF
04     for v in range(len(dist)) :
05         if not selected[v] and dist[v]<mindist :
06             mindist = dist[v]
07             minv = v
08     return minv
    
```



A	B	C	D	E	F	G
0	x	x	x	x	x	x
⑦	29	x	x	x	10	x
	29	x	x	27	⑩	x
	29	x	22	27		25
	29	12	⑫			18
	16	⑫				18
	⑩					15
						⑮

알고리즘: Prim의 MST



알고리즘 테스트

Prim의 MST 알고리즘

```
vertex = ['A',... ]           # 그림 8.12의 vertex
weight = [ ... ]              # 그림 8.12의 weight
print("MST By Prim's Algorithm")
MSTPrim(vertex, weight)
```

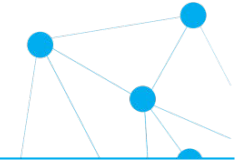
정점이 선택되는 과정 md.exe

```
MST By Prim's Algorithm
A: [0, 29, 9999, 9999, 9999, 10, 9999]
F: [0, 29, 9999, 9999, 27, 10, 9999]
E: [0, 29, 9999, 22, 27, 10, 25]
D: [0, 29, 12, 22, 27, 10, 18]
C: [0, 16, 12, 22, 27, 10, 18]
B: [0, 16, 12, 22, 27, 10, 15]
G: [0, 16, 12, 22, 27, 10, 15]
```

dist 배열 변화

- 시간 복잡도: $O(n^2)$

8.4 최소비용 신장트리: Kruskal



- 현재 선택할 수 있는 가장 가중치가 작은 간선을 선택
- 문제: 만약 MST에 사이클이 생긴다면?
 - 그 간선을 넣지 않아야 함.

알고리즘 8.6

Kruskal의 최소비용 신장트리 알고리즘(자연어)

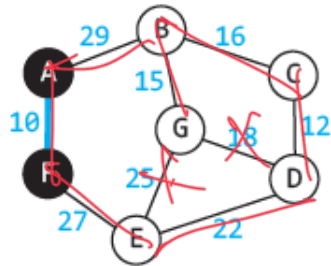
kruskal()

1. 그래프의 모든 간선을 가중치에 따라 오름차순으로 정렬한다.
2. 가장 가중치가 작은 간선 e 를 뽑는다.
3. e 를 신장트리에 넣었을 때 사이클이 생기면 넣지 않고 2번으로 이동한다.
4. 사이클이 생기지 않으면 최소 신장트리에 삽입한다.
5. $n-1$ 개의 간선이 삽입될 때 까지 2번으로 이동한다.

Kruskal의 탐욕

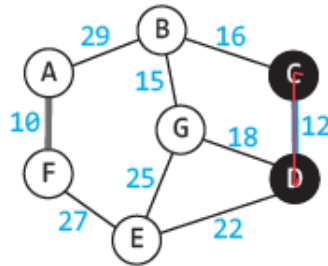
=> 그래프에서 최소 간선 선택
사이클을 제거 (MST)

최소 가중치 간선



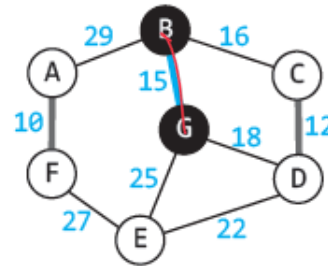
AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(a) AF 선택 → MST 추가



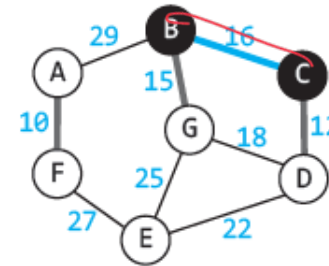
AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(b) CD 선택 → MST 추가



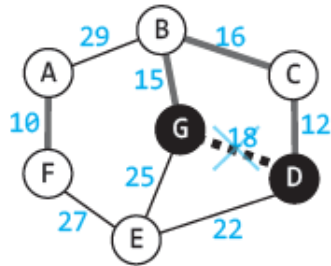
AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(c) BG 선택 → MST 추가



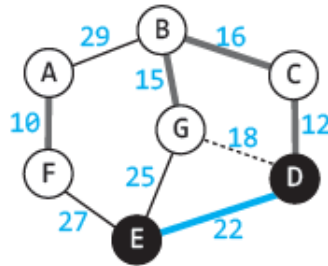
AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(d) BC 선택 → MST 추가



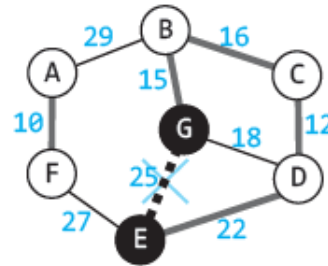
AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(e) DG 선택 → 사이클 발생



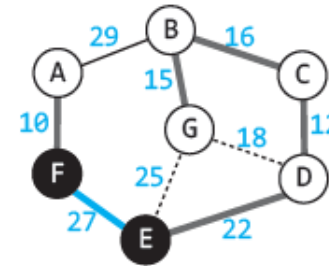
AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(f) DE 선택 → MST 추가



AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(g) EG 선택 → 사이클 발생

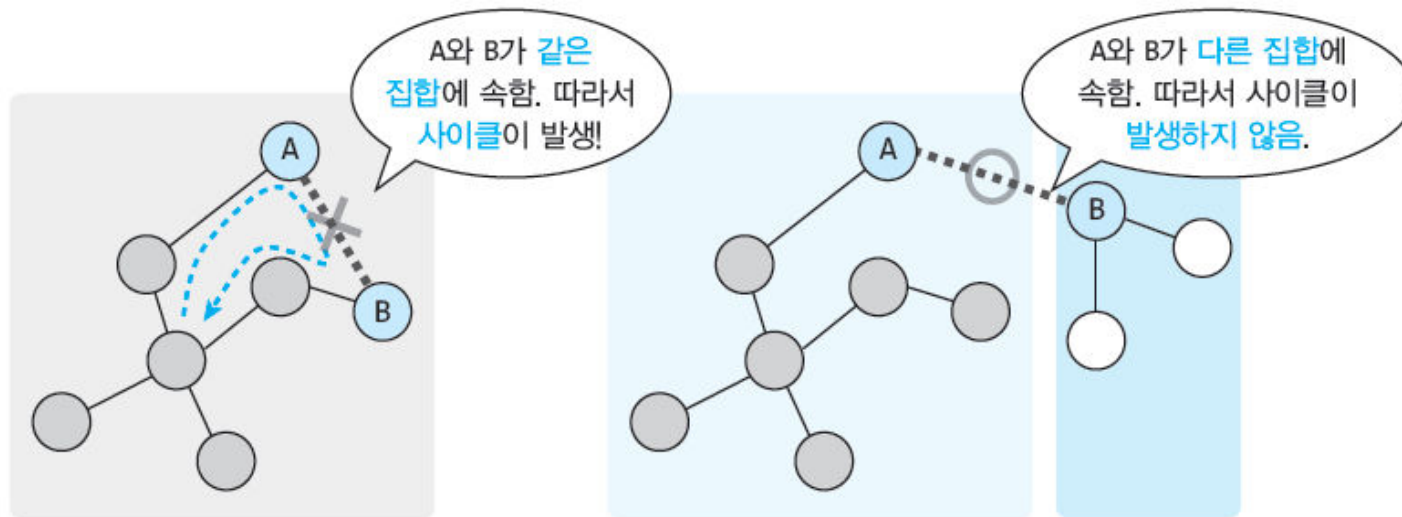


AF	CD	BG	BC	DG	DE	EG	EF	AB
10	12	15	16	18	22	25	27	29

(h) EF 선택 → MST 추가
MST 완성

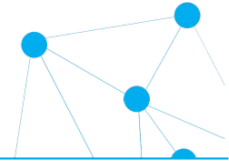
사이클 검사

- 새로운 간선이 사이클을 만드는 경우?
 - 이미 다른 경로를 통해 연결된 정점들 사이의 간선인 경우



- 해결 방안
 - 서로소 부분집합과 Union-Find 알고리즘
 - $\text{find}(x)$: 원소 x 가 속한 집합 $\rightarrow$ 보통은 "대표원소" 반환
 - $\text{union}(x,y)$: 원소 x 와 y 가 속한 집합을 하나로 뭉침

트리를 이용한 Union-Find 구현



- $S = \{1, 2, 3, 4, 5, 6\}$

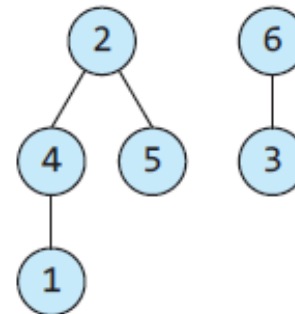
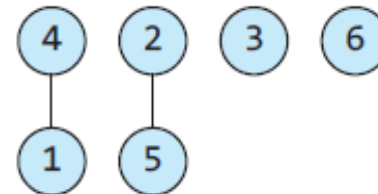
$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$

- $\text{union}(1, 4)$ 와 $\text{union}(5, 2)$

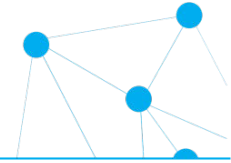
$\{1, 4\}, \{5, 2\}, \{3\}, \{6\}$

- $\text{union}(4, 5)$ 와 $\text{union}(3, 6)$

$\{1, 4, 5, 2\}, \{3, 6\}$



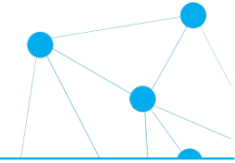
클래스 구현: disjointSets(서로소 집합)



클래스 8.1 disjointSets(서로소 집합)

```
01 class disjointSets:
02     def __init__(self, n):
03         self.parent = [-1]*n
04         self.set_size = n
05
06     def find(self, id) :
07         while (self.parent[id] >= 0):
08             id = self.parent[id]
09         return id
10
11     def union(self, s1, s2) :
12         self.parent[s1] = s2
13         self.set_size = self.set_size-1
```

알고리즘: Kruskal의 MST



알고리즘 8.7 Kruskal의 최소비용 신장트리

```
01 def MSTKruskal(V, adj):
02     n = len(V)
03     dsets = disjointSets(n)
04     E = []
05     for i in range(n-1) :
06         for j in range(i+1, n) :
07             if adj[i][j] != None :
08                 E.append((i,j,adj[i][j]))
09
10     E.sort(key= lambda e : e[2])
11
12     ecount = 0
13     for i in range(len(E)):
14         e = E[i]
15         uset = dsets.find(e[0])
16         vset = dsets.find(e[1])
17
18         if uset != vset :
19             dsets.union(uset, vset)
20             print("간선 추가 : (%s, %s, %s)" % (e[0], e[1], e[2]))
21             ecount += 1
22         if ecount == n-1 :
23             break
```

알고리즘 테스트 Kruskal의 MST 알고리즘

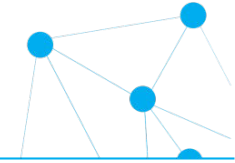
```
vertex = ['A',... ]      # 그림 8.12의 vertex
weight = [ ... ]         # 그림 8.12의 weight
print("MST By Kruskal's Algorithm")
MSTKruskal1(vertex, weight)
```

```
C:\WINDOWS\system32\cmd.exe
MST By Kruskal's Algorithm
간선 추가 : (A, F, 10)
간선 추가 : (C, D, 12)
간선 추가 : (B, G, 15)
간선 추가 : (B, C, 16)
간선 추가 : (D, E, 22)
간선 추가 : (E, F, 27)
```

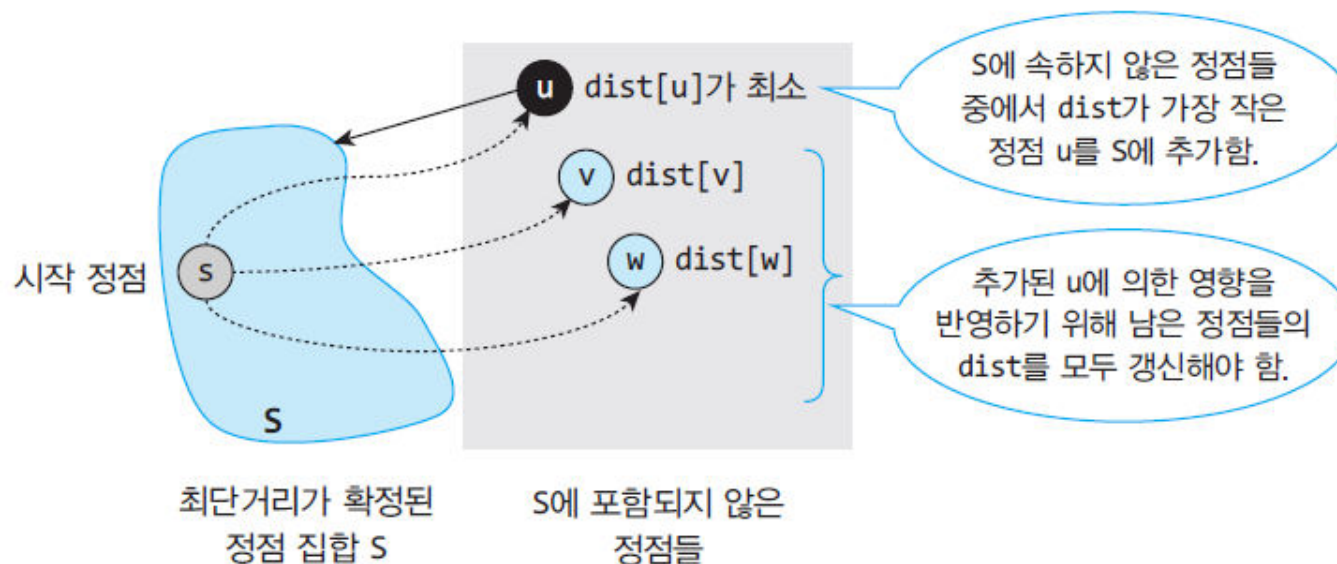
간선이 선택되는 과정

- 시간 복잡도(인접행렬)
 - 3-5행: $O(n^2)$
 - 10행: $O(e \log_2 e)$
 - $O(n^2 + e \log_2 e)$
- 인접 리스트 표현:
 - $O(e \log_2 e)$

8.5 Dijkstra의 최단경로 알고리즘

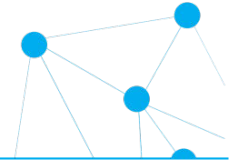


- 한 정점에서 다른 모든 정점까지의 최단 경로 거리
- Dijkstra가 선택한 탐욕
 - 최단거리가 확정된 정점들과 간선으로 직접 연결된 정점들 중에서 가장 가까운 정점을 선택한다.

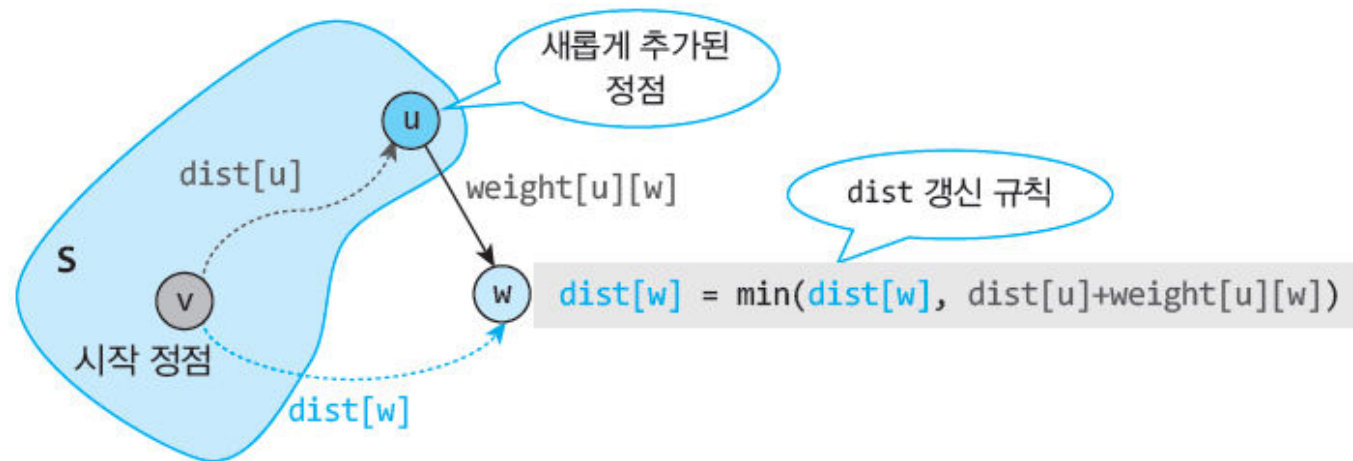


[그림 8.17] Dijkstra 최단경로 알고리즘의 기본 아이디어

Distance 갱신

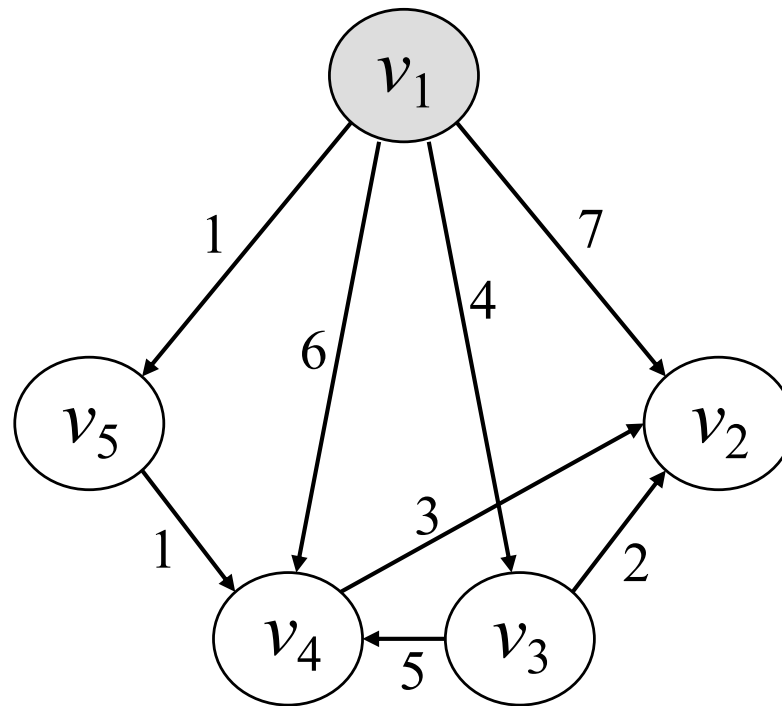


- 각 정점 w 에 대해
 - u 를 거쳐서 w 로 가는 새로운 경로가 더 짧다면 $\text{dist}[w]$ 갱신
$$\text{dist}[w] = \text{dist}[v] + \text{weight}[u][w]$$
 - 그렇지 않다면 $\text{dist}[w]$ 유지

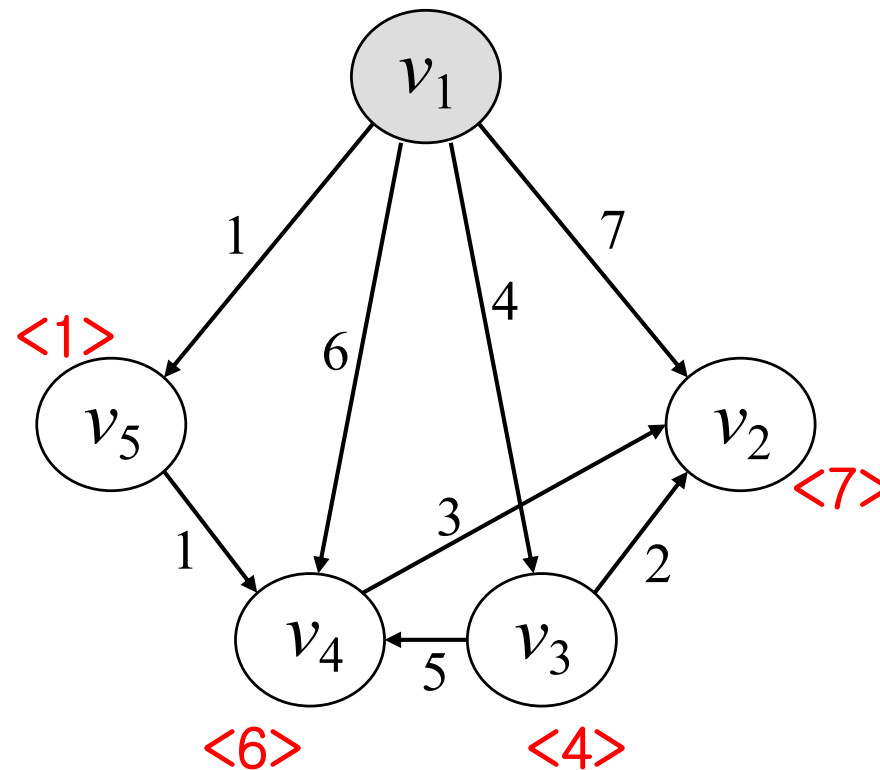


[그림 8.18] 최단경로 알고리즘에서의 distance 갱신

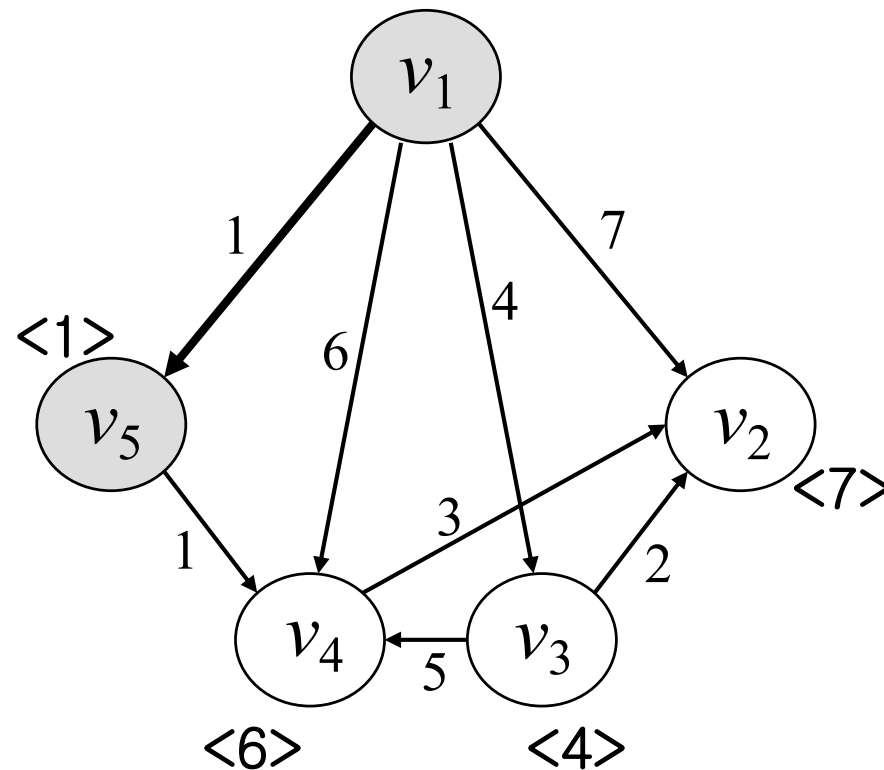
8.5 Dijkstra의 최단경로 알고리즘 실행 예



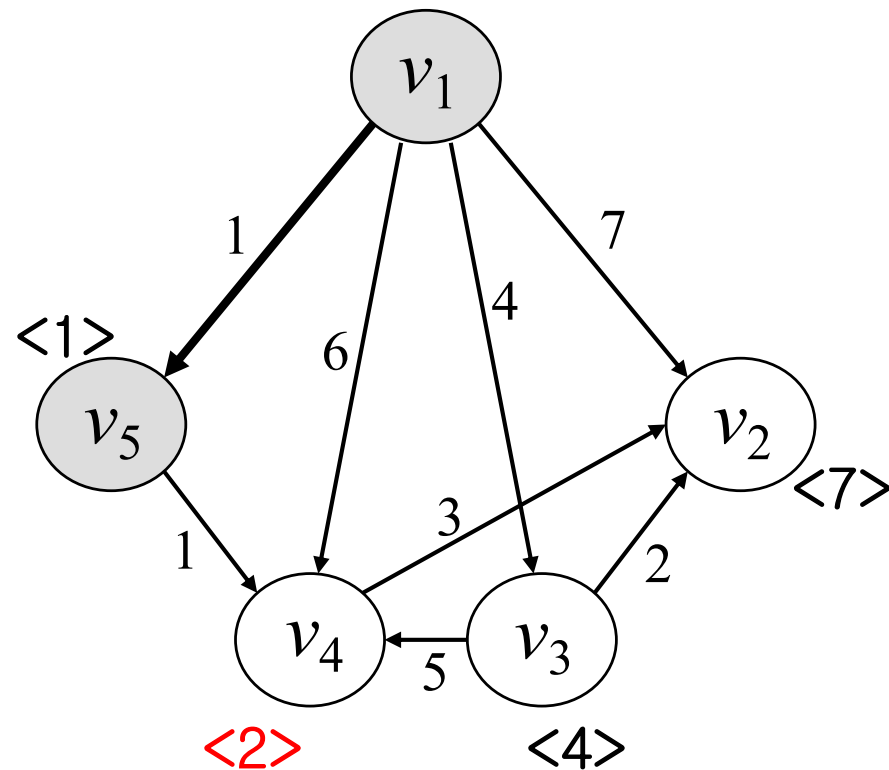
8.5 Dijkstra의 최단경로 알고리즘 실행 예



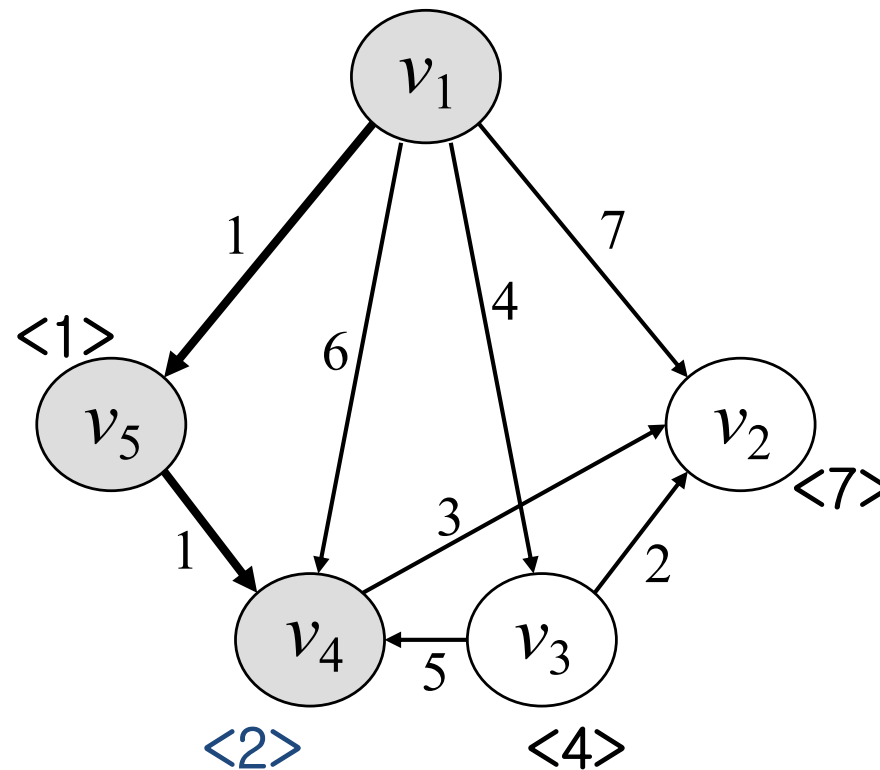
8.5 Dijkstra의 최단경로 알고리즘 실행 예



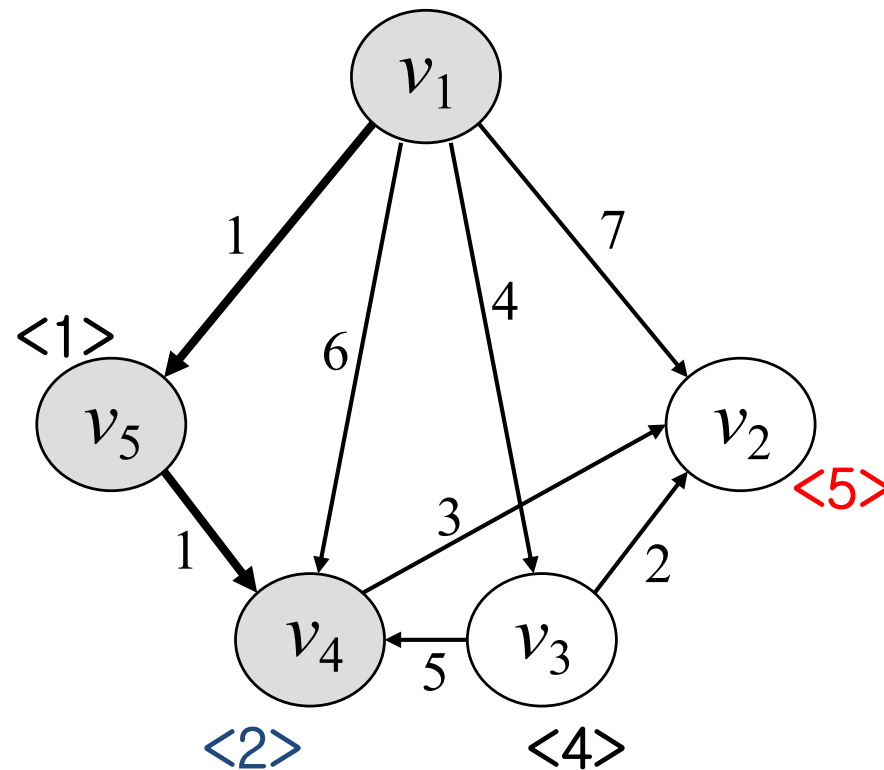
8.5 Dijkstra의 최단경로 알고리즘 실행 예



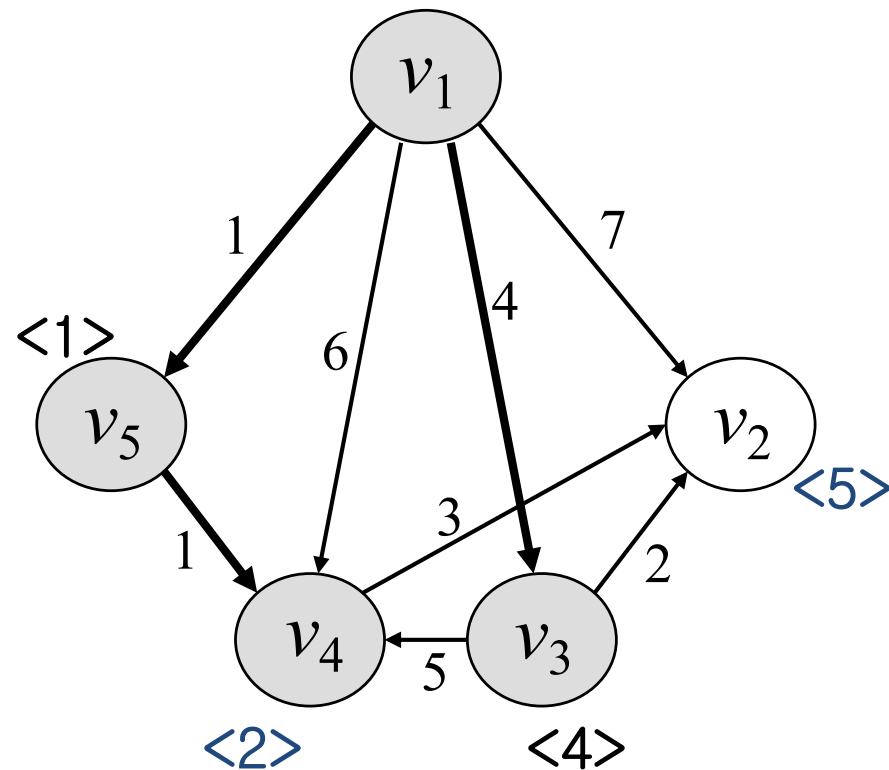
8.5 Dijkstra의 최단경로 알고리즘 실행 예



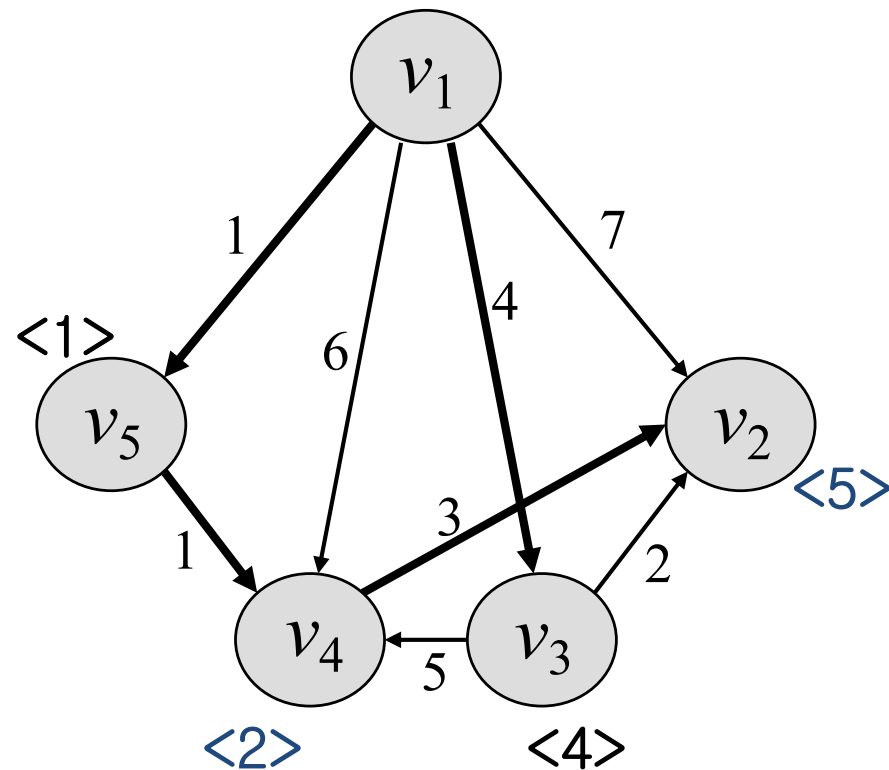
8.5 Dijkstra의 최단경로 알고리즘 실행 예



8.5 Dijkstra의 최단경로 알고리즘 실행 예



8.5 Dijkstra의 최단경로 알고리즘 실행 예



Dijkstra 알고리즘 진행 과정

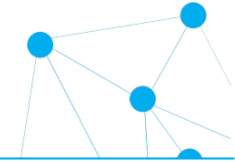
- A가 시작 정점인 경우



[그림 8.19] Dijkstra 알고리즘 진행 과정

- 탐욕적 선택 단계의 타당성
 - 심화학습: p351

알고리즘



C:\WINDOWS\system32\cmd.exe

Shortest Path By Dijkstra Algorithm

Step 1: [0, 7, 999, 999, 3, 10, 999]

Step 2: [0, 5, 999, 14, 3, 10, 8]

Step 3: [0, 5, 9, 14, 3, 10, 8]

Step 4: [0, 5, 9, 12, 3, 10, 8]

Step 5: [0, 5, 9, 11, 3, 10, 8]

Step 6: [0, 5, 9, 11, 3, 10, 8]

Step 7: [0, 5, 9, 11, 3, 10, 8]

[최단경로: A->B] B <- E <- A

[최단경로: A->C] C <- B <- E <- A

[최단경로: A->D] D <- C <- B <- E <- A

[최단경로: A->E] E <- A

[최단경로: A->F] F <- A

[최단경로: A->G] G <- E <- A

각 단계별 dist[] 배열의 값 변화

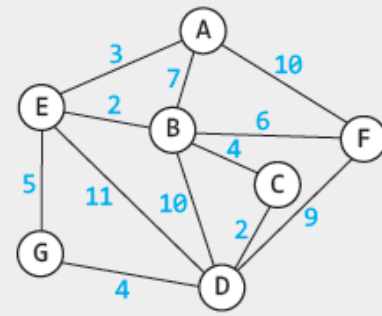
최종적인 최단경로 거리 (출발 정점: 0)

A부터 모든 정점까지의 최단경로

Floyd 알고리즘의 최종 A 행렬

0	5	9	11	3	10	8
5	0	4	6	2	6	7
9	4	0	2	6	10	6
11	6	2	0	8	9	4
3	2	6	8	0	8	5
10	6	10	9	8	0	13
8	7	6	4	5	13	0

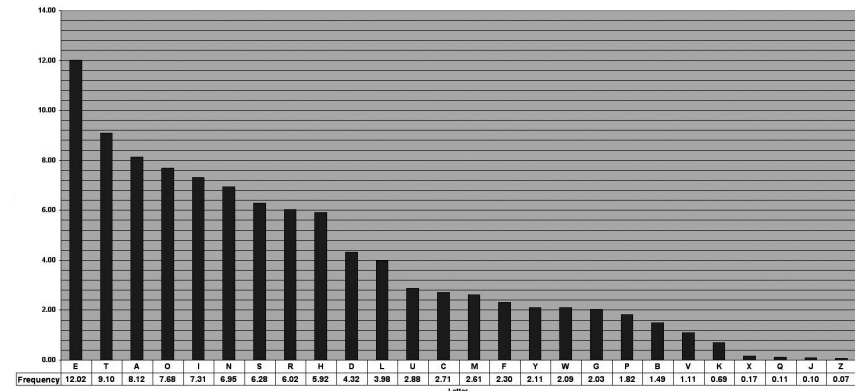
계속하려면 아무 키나 누르십시오 . . .



- 시간 복잡도: $O(n^2)$

8.6 허프만 코드

- 영어 알파벳별 빈도수 예

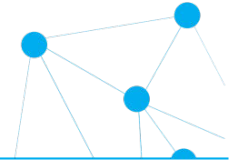


- 가변 길이 코드

빈도수 테이블		고정길이 코드			가변길이 코드		
문자	빈도수	코드	문자의 비트수	전체 비트수	코드	문자의 비트수	전체 비트수
A	24	000	3	72	00	2	48
B	3	001	3	9	11110	5	15
C	8	010	3	24	011	3	24
D	10	011	3	30	010	3	30
E	33	100	3	99	10	2	66
F	6	101	3	18	1110	4	24
G	4	110	3	12	11111	5	20
H	12	111	3	36	110	3	36
	100		24	300		27	263

[그림 8.21] 빈도수가 알려진 문자에 대한 고정길이 코드와 가변길이 코드의 비교

문자의 표현과 해독



- “FADE”의 인코딩

고정길이 코드:

F	A	D	E
101	000	011	100

가변길이 코드:

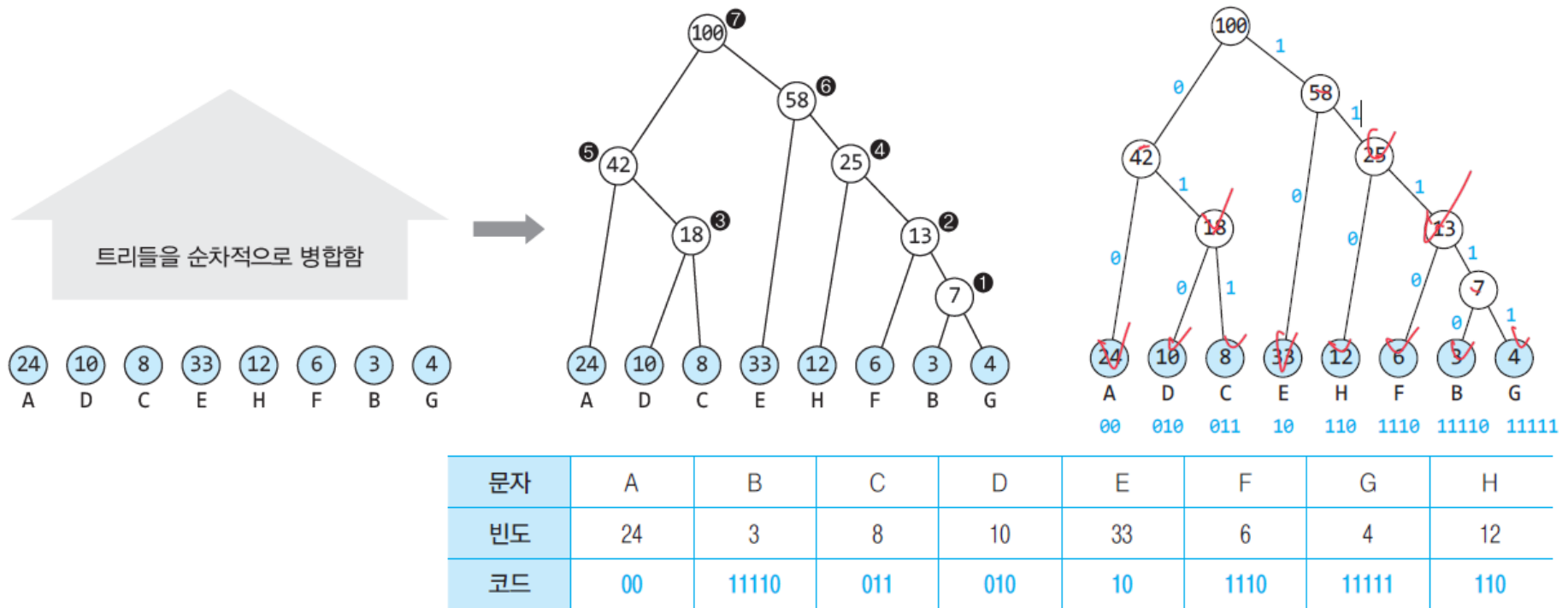
F	A	D	E
1110	00	010	10

모든 코드가 다른 코드의
첫 부분이 아니어야 함.
Prefix-free code

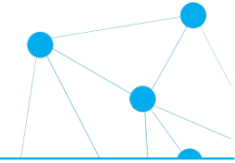
- 가변길이 코드 디코딩은?
 - 접두어 없는(prefix-free) 코드
 - 비트열을 왼쪽에서 오른쪽으로 조사하면 정확히 하나의 코드만 일치
- 빈도표 → 가변길이코드
 - 허프만 코드(Huffman codes)
 - 탐욕적 전략 사용

허프만 코드 생성

- 허프만 트리(Huffman tree)
 - 탐욕적인 해: 트리 중에서 루트의 값이 가장 두 트리 선택
- 허프만 트리 생성 과정 → 코드 부여



알고리즘: 허프만 트리 생성



알고리즘 8.9 허프만 트리 생성 알고리즘

```
01 import heapq
02 def make_heap_tree(freq, label):
03     n = len(freq)
04     h=[]
05     for i in range(n):
06         heapq.heappush(h, (freq[i], label[i]))
07
08     for i in range(1, n) :
09         e1 = heapq.heappop(h)
10         e2 = heapq.heappop(h)
11         heapq.heappush(h, (e1[0]+e2[0], e1[1]+e2[1]))
12         print(e1, "+", e2)
13
14     print(heapq.heappop(h))
```

알고리즘 테스트 허프만 트리 알고리즘

```
label = [ 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H' ]
freq = [24, 3, 8, 10, 33, 6, 4, 12]
make_heap_tree(freq, label)
```

C:\WINDOWS\system32\cmd.exe

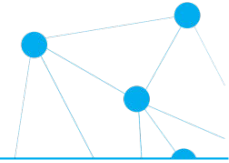
```
(3, 'B') + (4, 'G')
(6, 'F') + (7, 'BG')
(8, 'C') + (10, 'D')
(12, 'H') + (13, 'FBG')
(18, 'CD') + (24, 'A')
(25, 'HFBG') + (33, 'E')
(42, 'CDA') + (58, 'HFBGE')
(100, 'CDAHFBGE')
```

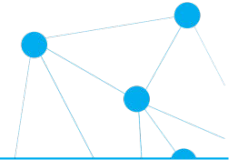
트리가 합해지는 과정

최종 트리의 루트

- 시간 복잡도(우선순위 큐 사용): $\rightarrow O(n \log_2 n)$

실습 과제





감사합니다!